Date: / /

GPU Programming:

GPU: (Hardware)

- Streaming Multiprocessors
  - ↳ Independant compute units.
  - ↓
  - Multiple CUDA cores.
    - ↳ Execute lockstep cores a.k.a WARPS (32 threads)
- Memory: Registers (Fastest)
  - ↓
  - Shared Memory (Fast, on-SM)
  - ↓
  - Global Memory (Slow, DRAM)

Software:

CUDA: → Write a kernel - fn. that runs on GPU.

You write threads' 12a work. That GPU runs 1000s of the same at once

→ Threads are grouped into blocks grid.

↓

- Share "shared Memory" & Sync w/E.O.
- Can't communicate accross blocks.

Work index = thread index + block dimention × block Index

1024 Elements

↳ Issue is that you can't launch 1024 Individually named threads.

So, GPU Organizes them in Blocks

Each Block's index starts from 0 → 255.

∴ thread_idx isn't unique.

So, Block 0 thread 5 & Block 1 thread 5 ~~would be pointed by the~~ ptr. would be considered the same.

$$\therefore i = \overbrace{\text{blockIdx} * \text{block_dim}}^{\text{Skips past all prev. blocks}} + \underbrace{\text{threadIdx.x}}_{\text{Pick the posn within the block.}}$$

B1T5 = $1 \times 256 + 5 = 261$

Why CudaMalloc:

↳ Physically, CPU & GPU's Memory is separate, So, CPU's Malloc does this process in RAM, which is directly inaccessible ~~to GP~~ via GPU.

So,

cudaMalloc allocates here

CPU RAM		GPU VRAM
h_a	$\xrightarrow{cpy}$	d_a
h_b	$\xrightarrow{cpy}$	d_b
		push result in d_c
h_c	$\xleftarrow{cpy}$	

Hence direction HostToDevice / DeviceToHost has to be specified clearly.

Date: / /

Matrix Multiplication:

Eg

$$A \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \qquad B \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

$A * B = C$

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}_{3\times 2} \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}_{2\times 3} = \begin{bmatrix} 9 & 12 & 15 \\ 19 & 26 & 33 \\ 29 & 40 & 51 \end{bmatrix}$$

Lets say $C[2][3]$ is what we need to figure out

So, we need 2nd row of A & 3rd column of B

loop k

→ ~~c+=r(A).c(B)~~ $s+ = r(A) \cdot c(B)$

In GPU, for vector Addition, we had 1D grid, But Mat_Mul is a 2D Case.

So,

for $C[2][3]$

row = blockIdx.y * block_dim(y) + ThreadIdx(y)
col = blockIdx(x) * block_dim(x) + ThreadIdx(x)

Same as 1D But row/col have their own threadPosn in